

Automaton Auditor — Forensic Audit Report

Author: Rahel Samson

Date: 2026-03-01

Audit Target: Abnet-Melaku1/automation-auditor

Overall Score: 3.8 / 5.0

1. Executive Summary

The Automaton Auditor is a hierarchical multi-agent system built on LangGraph's StateGraph that treats code evaluation as a legal proceeding. Three Detective agents (RepoInvestigator, DocAnalyst, VisionInspector) collect forensic evidence in parallel, pass it through a metacognitive synchronization barrier (EvidenceAggregator), and submit it to three Judge agents (Prosecutor, Defense, TechLead) with deliberately conflicting personas. A deterministic ChiefJustice synthesizes the final verdict using a rule-driven pipeline with no LLM call, no averaging, and no subjective interpretation.

Self-Audit Verdict: 3.8 / 5.0 across 9 rubric dimensions when executed against the peer repository Abnet-Melaku1/automation-auditor (<https://github.com/Abnet-Melaku1/automation-auditor>). Four dimensions scored a perfect 5/5 (Git Forensics, Graph Orchestration, Safe Tool Engineering, Structured Output Enforcement), demonstrating strong core architecture.

Most Impactful Peer Feedback Finding: During Week 1, peer agents revealed that our original regex-based StateGraph detection passed files that merely imported StateGraph without instantiating it, a false-positive flaw that could inflate scores for non-functional submissions. This led to a complete replacement with deep AST parsing via ast.NodeVisitor subclasses in src/tools/repo_tools.py.

Top Remaining Gap: The VisionInspector's diagram analysis is currently a simulated placeholder (confidence: 0.50). It cannot genuinely verify whether a PDF diagram accurately depicts parallel branches, making the "Architectural Diagram Analysis" dimension unreliable (scored 1/5 in the peer audit). This is the system's single biggest credibility gap.

Primary Remediation Priority: Integrate a real multimodal LLM call (GPT-4o Vision or Gemini Pro Vision) into VisionTools.analyze_diagram() to replace the simulation with actual diagram comprehension. This single change would address the lowest-scoring dimension and elevate the system's forensic completeness from 8/9 to 9/9 functional dimensions.

2. Architecture Deep Dive and Diagrams

2.1 Dialectical Synthesis — How Three Adversaries Produce One Verdict

Dialectical Synthesis is not a buzzword in this system it is the core judicial mechanism implemented across three files:

Thesis (Prosecutor) — `src/nodes/judges.py:Prosecutor()`

The system prompt instructs the model to “find weakness and technical debt. Be adversarial. Look for laziness, security flaws, and gaps.” A score of 5 is nearly impossible from the Prosecutor.

Antithesis (Defense) — `src/nodes/judges.py:Defense()`

The system prompt instructs the model to “highlight strengths and viable workarounds. Reward effort, intent, and progress.” The Defense actively argues against the Prosecutor’s penalties.

Synthesis (ChiefJustice) — `src/nodes/judges.py:ChiefJustice()`

This component does not re-prompt an LLM. Instead, it applies a 4-step deterministic pipeline:

- **Step 1 — Hard Caps:** SECURITY conflicts cap the dimension at 3; FACTCHECK conflicts set the Defense score to 1 (hallucination penalty).
- **Step 2 — Authority Rule:** For `graph_orchestration`, if `TechLead` ≥ 4 , the `TechLead` score becomes the sole final score.
- **Step 3 — Weighted Fallback:** $(\text{TechLead} \times 0.4) + (\text{Prosecutor} \times 0.3) + (\text{Defense} \times 0.3)$.
- **Step 4 — Clamp & Round:** `Decimal(ROUND_HALF_UP)` to integer 1–5 to avoid Python’s banker’s rounding behavior.

The conflict is real and structural. On the “Judicial Nuance” dimension of our peer audit, the Prosecutor scored 2/5 while the Defense scored 4/5 a genuine dialectical tension resolved by the weighted formula rather than by arbitrarily choosing a winner.

2.2 Fan-In / Fan-Out — Concrete Graph Edges

The system implements two distinct parallel patterns, each defined by explicit `builder.add_edge()` calls in `src/graph.py`.

Detective Fan-Out: After `ContextBuilder`, a conditional router dispatches to up to three detectives simultaneously.

```
builder.add_conditional_edges("ContextBuilder", start_router, {
    "RepoInvestigator": "RepoInvestigator",
    "DocAnalyst": "DocAnalyst",
    "VisionInspector": "VisionInspector",
    "failure_node": "failure_node"
})
```

Detective Fan-In:

```
builder.add_edge("RepoInvestigator", "EvidenceAggregator")
builder.add_edge("DocAnalyst", "EvidenceAggregator")
builder.add_edge("VisionInspector", "EvidenceAggregator")
```

Judge Fan-Out:

```
builder.add_edge("EvidenceAggregator", "Prosecutor")
builder.add_edge("EvidenceAggregator", "Defense")
builder.add_edge("EvidenceAggregator", "TechLead")
```

Judge Fan-In:

```
builder.add_edge("Prosecutor", "ChiefJustice")
builder.add_edge("Defense", "ChiefJustice")
builder.add_edge("TechLead", "ChiefJustice")
```

2.3 Metacognition — The System Evaluates Its Own Evaluation

Metacognition is implemented through the EvidenceAggregator node, which acts as a forensic firewall between the Detective and Judicial layers. Before any judge sees evidence, the aggregator cross-references what the DocAnalyst claimed against what the RepoInvestigator actually found.

```
for doc_e in doc_ev:
    repo_match = next((r for r in repo_ev if r.goal == doc_e.goal), None)
    if doc_e.found and (not repo_match or not repo_match.found):
        conflicts.append(ConflictEntry(
            tag="FACTCHECK",
            dimension_id=dim_id,
            message="PDF claim not supported by repository evidence."
        ))
```

If a PDF claims “We implemented parallel Judges” but the repository shows a linear graph, the FACTCHECK conflict prevents the Defense from inflating the score based on unverified claims. In our peer audit, this mechanism caught two hallucinated claims and triggered the fact_supremacy rule.

2.4 Data Flow: Evidence → Opinions → Verdict

AgentState flows through the pipeline via three reducer-protected channels:

- **evidences (Dict[str, List[Evidence]])**
Reducer: operator.ior (dictionary merge)
Detectives write to disjoint keys. Parallel writes merge without overwriting.
- **opinions (List[JudicialOpinion])**
Reducer: operator.add (list concatenation)
Judges append opinions independently; lists concatenate safely.
- **conflict_log (List[ConflictEntry])**
Reducer: operator.add
EvidenceAggregator emits FACTCHECK and SECURITY entries.

The ChiefJustice reads all three channels and applies the deterministic 4-step synthesis pipeline. Reducers ensure that no parallel branch overwrites another.

2.5 StateGraph Flow Overview

START

→ ContextBuilder (loads rubric.json — 9 dimensions, 4 synthesis rules)
→ Parallel Detectives (RepolInvestigator, DocAnalyst, VisionInspector)
→ EvidenceAggregator (metacognitive barrier, FACTCHECK and SECURITY scan)
→ Parallel Judges (Prosecutor, Defense, TechLead)
→ ChiefJustice (Hard Caps → Authority → Weighted → Clamp)
→ END
→ AuditReport (3.8/5.0)
→ reports/forensic_report.md

2.6 Design Trade-offs

Pydantic + Annotated reducers over raw dicts:

Raw dictionaries allow silent schema errors. Pydantic enforces structure. Annotated reducers ensure safe parallel merging. Trade-off: additional boilerplate for increased safety.

Deterministic synthesis over LLM averaging:

Deterministic rules provide explainable and reproducible verdicts. Trade-off: slightly less nuanced than an LLM mediator but fully auditable.

AST parsing over regex:

Regex cannot distinguish between import statements and actual instantiation. AST parsing eliminates false positives. Trade-off: slightly higher compute cost per file.

RAG-lite keyword search over embeddings:

Keyword search is deterministic and avoids hallucinated semantic matches. Trade-off: reduced semantic flexibility.

tempfile sandbox over local clone:

Ensures forensic isolation and automatic cleanup. Trade-off: minor I/O overhead per audit.

3. Self-Audit Criterion Breakdown

Overall Score: 3.8 / 5.0 (9 dimensions, 2 conflicts)

3.1 Git Forensic Analysis — 5/5

Clear atomic commit progression. Unanimous judge agreement.

3.2 Pydantic State Modeling — 4/5

Strong reducer usage. Minor stylistic divergence (BaseModel vs TypedDict).

3.3 Graph Orchestration Architecture — 5/5

Authority rule applied. TechLead score governs architecture dimension.

3.4 Safe Tool Engineering — 5/5

tempfile sandbox + subprocess.run + no unsafe calls.

3.5 Structured Output Enforcement — 5/5

All judges use `.with_structured_output(JudicialOpinion)`.

3.6 Judicial Nuance and Dialectics — 3/5

Persona prompts distinct but not aggressively adversarial enough.

3.7 Metacognition & Dialectic Depth — 3/5

FACTCHECK conflict triggered. Defense penalized for unverified claim.

3.8 Report Accuracy — 3/5

FACTCHECK triggered due to insufficient corroboration beyond path existence.

3.9 Architectural Diagram Analysis — 1/5

No verified StateGraph diagram. VisionInspector currently simulated.

4. MinMax Feedback Loop Reflection

Peer audits uncovered:

- Regex-based false positives
- Silent state overwrites without reducers
- os.system shell injection vulnerability
- Lack of metacognitive cross-referencing

Each issue was corrected with concrete code changes including AST visitors, reducer enforcement, subprocess sandboxing, and EvidenceAggregator implementation.

The systemic insight: detection capability mirrors architectural maturity. Every flaw detected in peers must also be guarded against internally.

5. Remediation Plan

Priority 1 — VisionInspector Multimodal Upgrade

Replace simulated diagram classification with GPT-4o Vision or Gemini Pro Vision.

Priority 2 — Deeper Persona Prompt Engineering

Expand persona prompts to increase adversarial depth and scoring granularity.

Priority 3 — Broader Context Gathering

Expand RepoInvestigator file scanning beyond keyword-based filtering.

Priority 4 — Retry Logic on Judge LLM Calls

Add tenacity-based retry with exponential backoff.

Priority 5 — Enhanced PDF Cross-Referencing

Add fuzzy matching and file path extraction for stronger FACTCHECK precision.

Audit executed: 2026-03-01

Target: Abnet-Melaku1/automation-auditor

Pipeline: LangGraph StateGraph with GPT-4o

Overall Score: 3.8 / 5.0